

TERRAFORM MODULES

- Modules are containers for multiple resources that are used together.
- A module consists of the collection of **.tf** and/or **.tf.json** files kept together in a directory.
- Modules are the main way to package and reuse resource configurations with terraform.

Real-Life Scenario: Deploying an AWS VPC

Imagine you need to deploy **multiple VPCs** (for dev, staging, and prod environments) with the same structure but different configurations (e.g., CIDR blocks, subnets).

Steps Without Modules

You would have to write the same Terraform code repeatedly for each environment, making it harder to maintain and prone to errors.

Solution With Modules

You can create a **VPC module** with reusable code and call it for each environment with different parameters.

Structure of the minimal module

```
$ tree minimal-module/  
.  
├── README.md  
├── main.tf  
├── variables.tf  
└── outputs.tf
```

README.md:

A text file that provides **documentation** for the module.

Explains the module's purpose, usage, required inputs, and outputs.

main.tf:

The core Terraform configuration file.

Defines the **resources**, **data sources**, and **logic** for the module.

variables.tf:

Specifies the **inputs** the module needs.

Defines variables with types, descriptions, and default values.

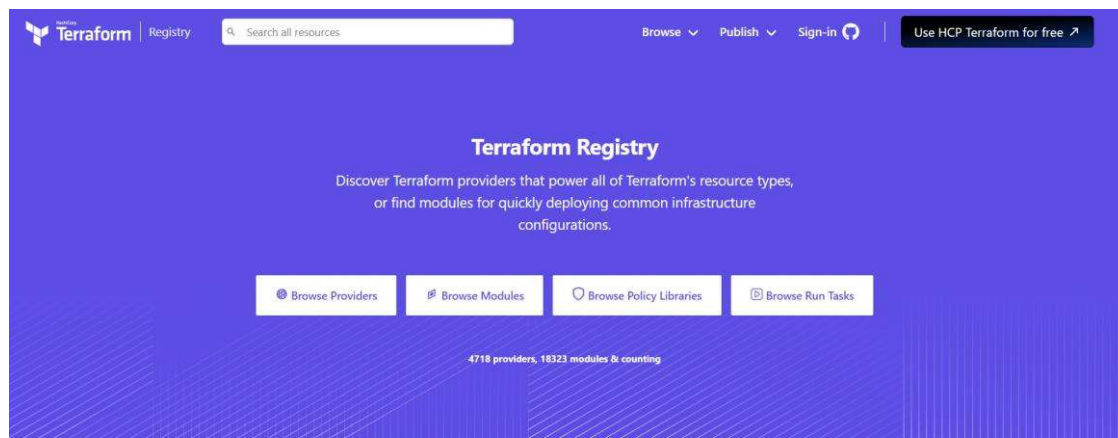
outputs.tf:

Declares the **outputs** from the module.

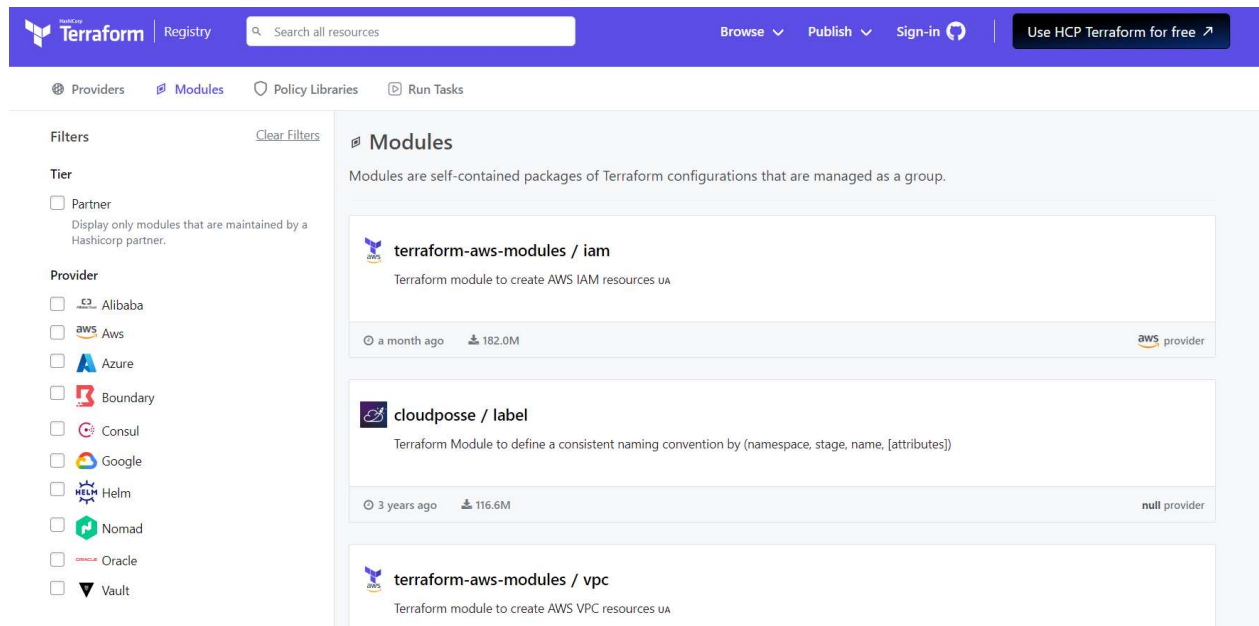
Shares resource information (like IDs, IPs) with the root module or other modules.

This structure ensures that the module is **self-contained**, reusable, and easy to understand. Let me know if you'd like a detailed explanation for any of these files!

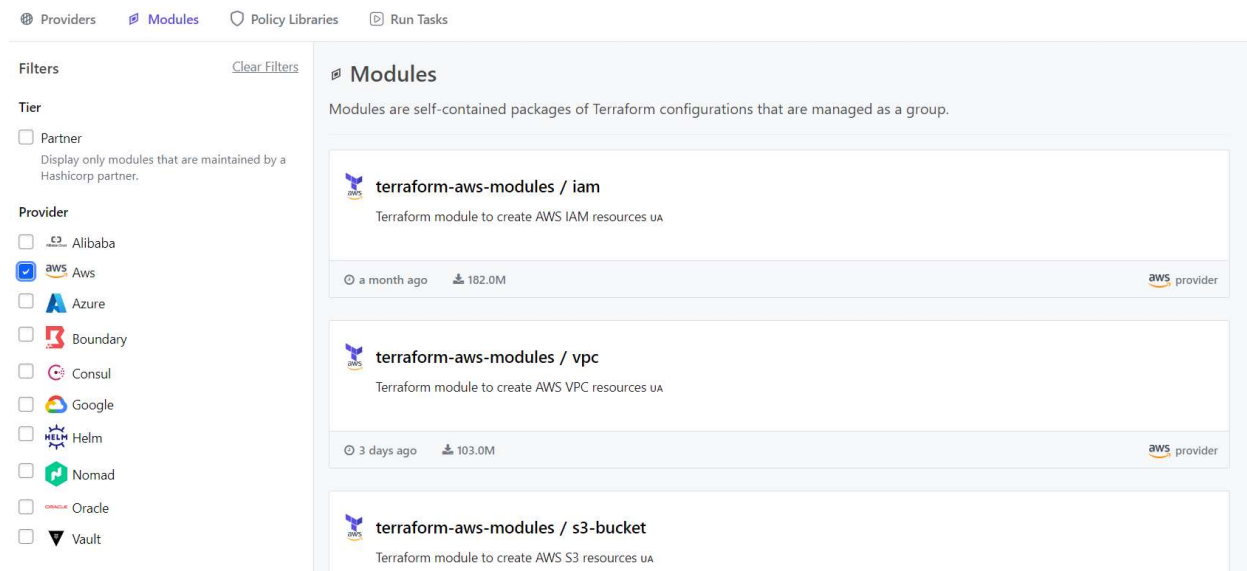
To get the proper modules go to “<https://registry.terraform.io>” you will see the something like this:



Now click on modules you will get to see something like this:



By selecting the AWS provider, you can access a wide range of modules tailored to various AWS services. These modules are organized with user-friendly names, making it easier to identify and choose the module that best fits your requirements.



Implementation of VPC using terraform modules

Let's implement a VPC using Terraform modules. We'll create a new directory named "**tf-module-vpc**". Inside this directory, we'll create a file called **compute.tf**. Next, search for the **AWS VPC module** in the Terraform Registry. You'll find a module that looks something like this:



vpc
aws provider

Terraform module to create AWS VPC resources ua

Published December 19, 2024 by terraform-aws-modules
Module managed by antonbabenko
Source Code: github.com/terraform-aws-modules/terraform-aws-vpc (report an issue)

Submodules Examples

Version 5.17.0 (latest)

Module Downloads All versions

Downloads this week	998,164
Downloads this month	2.9M
Downloads this year	41.5M
Downloads over all time	103.0M

Provision Instructions

Copy and paste into your Terraform configuration, insert the variables, and run `terraform init`:

```

module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.17.0"
}

```

Now copy the instruction that is given in “Provision instructions” in `compute.tf`

tf-module-vpc

compute.tf

```

AWS > tf-module-vpc > compute.tf > module "vpc"
Pieces: Comment | Pieces: Explain
1 module "vpc" {
2   source = "terraform-aws-modules/vpc/aws"
3   version = "5.17.0"
4 }

```

Now type “`terraform init`” in terminal, then you will notice something like:

tf-module-vpc

.terraform

modules

vpc

> .github

> examples

> modules

.editorconfig

.gitignore

! .pre-commit-config.yaml

.releaserc.json

CHANGELOG.md

LICENSE

main.tf

outputs.tf

README.md

UPGRADE-3.0.md

UPGRADE-4.0.md

variables.tf

versions.tf

vpc-flow-logs.tf

modules.json

providers\registry.terraform.io\has...

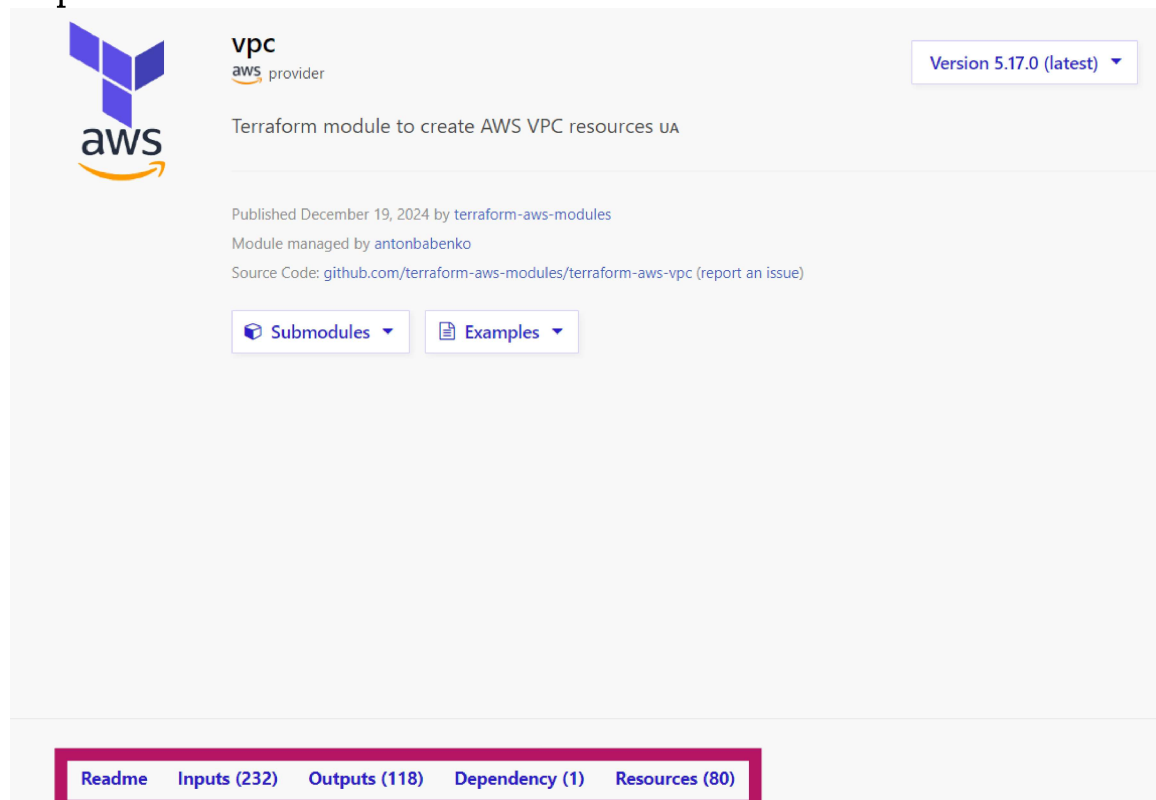
LICENSE.txt

terraform-provider-aws_v5.82.2_x5...

.terraform.lock.hcl

Go through “`main.tf`”, “`outputs.tf`”, “`variables.tf`”, “`versions.tf`”, “`vpc-flow-logs.tf`”

To understand how to use the module effectively, read the documentation thoroughly. Navigate to the required module in the Terraform Registry and scroll down. You'll find key sections like **"Readme"**, **"Inputs"**, **"Outputs"**, **"Dependencies"**, and **"Resources"**. These sections provide detailed information about the module's usage, configuration, required variables, outputs, and dependencies.



In the **"Inputs"** section of the Terraform Registry, you'll see all the variables defined in the module's variables.tf file. These variables automatically appear there for easy reference.

If you want to explore the infrastructure code directly, click on the **"Source Code"** link in the documentation. This will take you to the module's GitHub repository, where you can view all the details related to the AWS VPC provider.

When a module is implemented, it needs to know which AWS region to deploy the infrastructure in. To specify this, you provide the AWS region in the provider block or as a variable in your Terraform configuration.

```

1 provider "aws" {
2   region = "us-east-1"
3 }
4 module "vpc" {
5   source = "terraform-aws-modules/vpc/aws"
6   version = "5.17.0"
7 }

```

For further configuration do follow readme file of the module

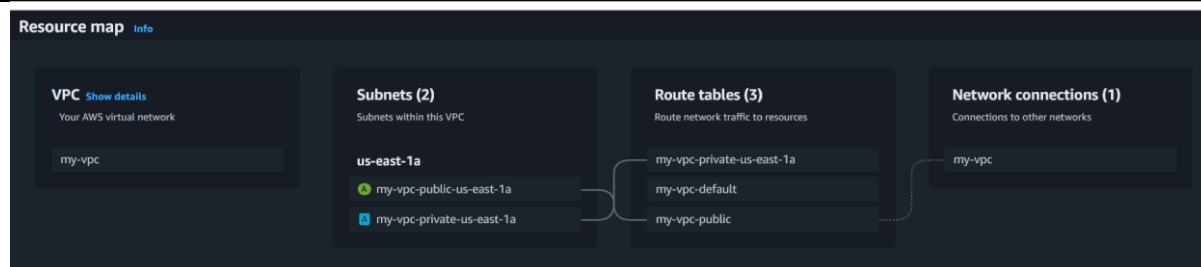
Final configuration of "compute.tf"

```

1 provider "aws" {
2   region = "us-east-1"
3 }
4 module "vpc" {
5   source = "terraform-aws-modules/vpc/aws"
6   version = "5.17.0"
7
8   name = "my-vpc"
9   cidr = "10.0.0.0/16"
10
11  azs          = ["us-east-1a"]
12  private_subnets = ["10.0.1.0/24"]
13  public_subnets  = ["10.0.101.0/24"]
14
15
16
17  tags = {
18    Terraform = "true"
19    Environment = "dev"
20  }
21 }
22

```

output



Implementation of EC2 instance using terraform modules

Now, we will create a new file named **instance.tf** inside the **tf-module-vpc** directory.

This file will be used to define EC2 instances.

To find the module for deploying EC2 instances, search for **terraform-aws-modules/ec2-instance** in the Terraform Registry.

Use the same process you followed to find the VPC module:

1. Go to the **Terraform Registry**.
2. Search for **terraform-aws-modules/ec2-instance**.
3. Review the module documentation for "**Readme**", "**Inputs**", "**Outputs**", "**Dependencies**", and "**Resources**" sections to understand its configuration and usage.

In "**instance.tf**"

```
1  module "ec2-instance" {
2      source = "terraform-aws-modules/ec2-instance/aws"
3      version = "5.7.1"
4
5      name = "single-instance"
6
7      ami = "ami-0453ec754f44f9a4a"
8
9      instance_type = "t2.micro"
10     vpc_security_group_ids = [module.vpc.default_security_group_id]
11     subnet_id = module.vpc.public_subnets[0]
12
13     tags = {
14         Name = "trial-instance"
15         Environment = "dev"
16     }
17 }
18 }
```

VPC Security Group and Subnet:

- vpc_security_group_ids: Associates the instance with the default security group from the **VPC module** (defined as module.vpc.default_security_group_id).

- `subnet_id`: Places the EC2 instance in the first public subnet created by the **VPC module** (`module.vpc.public_subnets[0]`).

Output

```
VPC ID
vpc-04587757c585bbcae (my-vpc)

Subnet ID
subnet-00b2649b5038a1c3b (my-vpc-public-us-east-1a)
```

NOTE: we are only doing “terraform plan” not “terraform apply” because it may cost you, do it at your own risk.

Building own terraform modules

Building your own Terraform modules involves creating reusable configurations to manage infrastructure more efficiently.

These modules group related resources together, making your code modular, scalable, and easy to maintain.

Custom **modules** are especially useful when standardizing infrastructure across multiple environments (e.g., dev, staging, prod).

A **modules** typically includes:

- **main.tf**: Defines resources.
- **variables.tf**: Specifies input variables.
- **outputs.tf**: Declares output values.

README.md: Helps users understand the module's purpose, how to use it.

Real-Life Scenario:

Scenario: A company needs to deploy VPCs with consistent configurations, so we need this all things in “tf-own-module”

Requirements:

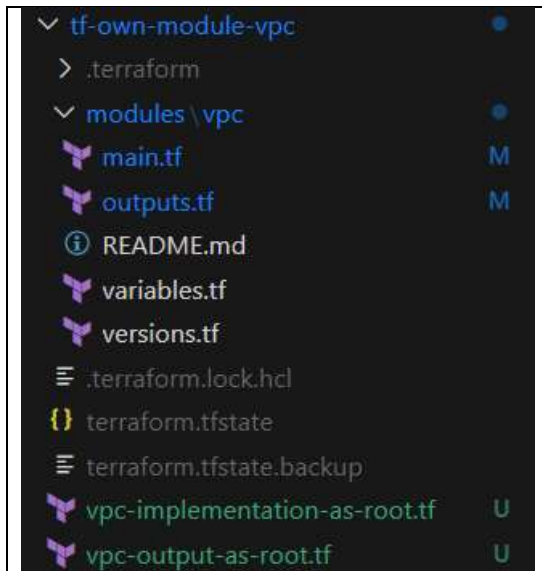
- Accept cidr_block from user to create VPC
- User can create multiple subnets
 - Get CIDR block for subnets from user
 - Get AZS (availability zone)
 - User can mark a subnet as public (default is private)
 - If public, create IGW
 - Associate public subnet with Routing table

We will create the following VPC configuration by developing a custom Terraform module.



Where all the subnets i.e. public or private is attached to Route table (`rtb-001538132f80805c1`) and this route table is attached to internet gateway

The file structure will be as follows:



In tf-own-module-vpc/modules/vpc/version.tf

```
1 terraform {
2   required_version = ">= 1.0.0"
3
4   required_providers {
5     aws = {
6       source = "hashicorp/aws"
7       version = ">= 4.0.0"
8     }
9   }
10 }
```

In tf-own-module-vpc/modules/vpc/variables.tf

```
1 variable "vpc_config" {
2   description = "To get the CIDR and the name of the vpc from user"
3   type = object({
4     cidr_block = string
5     vpc_name = string
6   })
7
8   validation {
9     condition = can(cidrnetmask(var.vpc_config.cidr_block))
10    error_message = "Each cidr_block must be a valid CIDR format."
11  }
12 }
13
```

```

14 Pieces: Comment | Pieces: Explain
variable "aws_subnets" {
15   # sub1={cidr_block, availability_zone}, sub2={}, sub3={}
16   description = "To get the CIDR and availability zone from user"
17   type = map(object({
18     cidr_block = string
19     availability_zone = string |
20     public = optional(bool, false)
21   }))
22
23 Pieces: Comment | Pieces: Explain
validation {
24   #sub1={cidr=...}, sub2={cidr=...}, sub3={cidr=...} , [true, true, true]
25   condition= alltrue([for config in var.aws_subnets : can(cidrnetmask(config.cidr_block))])
26   error_message = "Each cidr_block must be a valid CIDR format."
27 }
28 }

```

In tf-own-module-vpc/modules/vpc/main.tf

```

1  ✓ resource "aws_vpc" "vpc-main" {
2    cidr_block = var.vpc_config.cidr_block
3
4  ✓   tags = {
5     Name = var.vpc_config.vpc_name
6   }
7 }
8
9 Pieces: Comment | Pieces: Explain
resource "aws_subnet" "subnets_main" {
10   vpc_id = aws_vpc.vpc-main.id
11   for_each = var.aws_subnets
12
13   cidr_block = each.value.cidr_block
14   availability_zone = each.value.availability_zone
15
16   tags = {
17     Name = each.key
18   }
19 }
20
21 Pieces: Comment | Pieces: Explain
22 ✓ locals {
23   ✓ public_subnets = {
24     #public_sub1 = {cidr_block = "10.0.1.0/24", availability_zone = "us-east-1a", public = true}
25     for key, config in var.aws_subnets : key => config if config.public
26   }
27   ✓ private_subnets = {
28     for key, config in var.aws_subnets : key => config if !config.public
29   }
30 }

```

```

31
32 #internet gateway, if there is atleast one public subnet
33 #internet gateway will be created only once
    Pieces: Comment | Pieces: Explain
34 resource "aws_internet_gateway" "igw_main" {
35     vpc_id = aws_vpc.vpc-main.id
36     count = length(local.public_subnets) > 0 ? 1 : 0
37 }
    Pieces: Comment | Pieces: Explain
39 resource "aws_route_table" "route_main" {
40     vpc_id = aws_vpc.vpc-main.id
41     count = length(local.public_subnets) > 0 ? 1 : 0
42
    Pieces: Comment | Pieces: Explain
43     route {
44         cidr_block = "0.0.0.0/0"
45         gateway_id = aws_internet_gateway.igw_main[0].id
46     }
47 }
49 resource "aws_route_table_association" "route_table_association_main" {
50     for_each = local.public_subnets
51     subnet_id = aws_subnet.subnets_main[each.key].id
52     route_table_id = aws_route_table.route_main[0].id
53 }
54

```

In tf-own-module-vpc/modules/vpc/output.tf

```

1  output "vpc_id" {
2      description = "diplay the vpc id"
3      value = aws_vpc.vpc-main.id
4  }
5
6  locals {
7      #output format of subnets ids is subnet_name = {subnet_id, availability_zone}
8      public_subnet_output={
9          for key , config in local.public_subnets : key => {
10              subnet_id = aws_subnet.subnets_main[key].id
11              availability_zone = aws_subnet.subnets_main[key].availability_zone
12          }
13      }
14
15      private_subnet_output={
16          for key , config in local.private_subnets : key => {
17              subnet_id = aws_subnet.subnets_main[key].id
18              availability_zone = aws_subnet.subnets_main[key].availability_zone
19          }
20      }
21  }

```

```

23 output "public-subnet-ids" {
24     description = "list of public subnet ids"
25     value = local.public_subnet_output
26 }
27
28 Needs comment, Needs Explain
29 output "private-subnet-ids" {
30     description = "list of private subnet ids"
31     value = local.private_subnet_output
32 }

```

In tf-own-module-vpc/ vpc-implementation-as-root.tf

```

1  ✓ provider "aws" {
2      region = "us-east-1"
3  }
4
5  module "vpc" {
6      source = "../modules/vpc"
7
8      vpc_config = {
9          cidr_block = "10.0.0.0/16"
10         vpc_name = "implementing-vpc-using-own-module"
11     }
12
13     aws_subnets = {
14         public_sub1 = {
15             cidr_block = "10.0.1.0/24"
16             availability_zone = "us-east-1a"
17             public = true
18         }
19         public_sub2 = {
20             cidr_block = "10.0.2.0/24"
21             availability_zone = "us-east-1b"
22             public = true
23         }
24         private_sub3 = {
25             cidr_block = "10.0.3.0/24"
26             availability_zone = "us-east-1c"
27             public = false
28         }
29     }
30 }

```

In tf-own-module-vpc/ vpc-output-as-root.tf

```

1  Needs comment, Needs Explain
2  output "vpc_id" {
3      value = module.vpc.vpc_id
4  }

```



```

5   output "public-subnet-ids" {
6     value = module.vpc.public-subnet-ids
7   }
8
9   output "private-subnet-ids" {
10    value = module.vpc.private-subnet-ids
11  }

```

When you do *“terraform init”* → *“terraform plan”* → *“terraform apply”*
output

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

```

private-subnet-ids = {
  "public_sub1" = {
    "availability_zone" = "us-east-1a"
    "subnet_id" = "subnet-0ad6830dd993231b7"
  }
  "public_sub2" = {
    "availability_zone" = "us-east-1b"
    "subnet_id" = "subnet-014f0a99399cc0c94"
  }
}
public-subnet-ids = {
  "public_sub1" = {
    "availability_zone" = "us-east-1a"
    "subnet_id" = "subnet-0ad6830dd993231b7"
  }
  "public_sub2" = {
    "availability_zone" = "us-east-1b"
    "subnet_id" = "subnet-014f0a99399cc0c94"
  }
}
vpc_id = "vpc-0dffffdeb4b1b7e13c"

```

PREPARE MODULE FOR PUBLISH

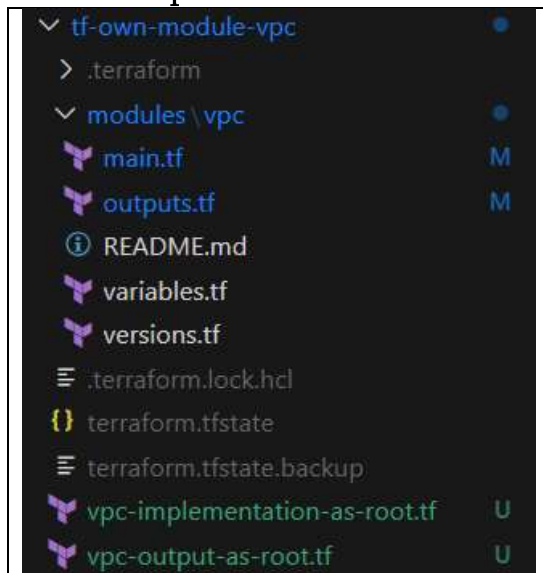
Creating your own Terraform module simplifies infrastructure management by enabling reusability, consistency, and scalability.

It allows you to define resources once and reuse them across projects, ensuring uniform configurations, reducing duplication, and making collaboration easier.

- Readme.md file
- LICENSE
- Example
- Push the code in GitHub
- Terraform Registry

To make this section successful we will take “tf-own-module” and add some extra files and directory.

This is a previous folder structure



In *tf-own-module-vpc/modules/vpc/ README.md*

```
1  #terraform-aws-vpc
2  | Ctrl+L to chat, Ctrl+K to generate
3  ▼ ## Overview
4
5  This Terraform module creates an AWS VPC with a given CIDR block.
6  It also creates multiple subnets (public and private), and for
7  public subnets, it sets up an Internet Gateway (IGW)
8  and appropriate route tables.
9
10 ▼ ## Features
11
12 - Creates a VPC with a specified CIDR block
13 - Creates public and private subnets
14 - Creates an Internet Gateway (IGW) for public subnets
15 - Sets up route tables for public subnets
16
17 ▼ ## Usage
18
19 ▼ ```
20 module "vpc" {
21   source = "../modules/vpc"
22
23   vpc_config = {
24     cidr_block = "10.0.0.0/16"
25     vpc_name = "test_vpc"
26   }
27
28   aws_subnets = {
29     public_sub1 = {
30       cidr_block = "10.0.1.0/24"
31       availability_zone = "us-east-1a"
32       # To set the subnet as public, default is private
33       public = true
34     }
35     public_sub2 = {
36       cidr_block = "10.0.2.0/24"
37       availability_zone = "us-east-1b"
38       public = true
39     }
40     private_sub3 = {
41       cidr_block = "10.0.3.0/24"
42       availability_zone = "us-east-1c"
43       public = false
44     }
45   }
46 }
47 ```
```

The code provided in the "Usage" section is extracted from ***tf-own-module-vpc/vpc-implementation-as-root.tf***, excluding the provider section.

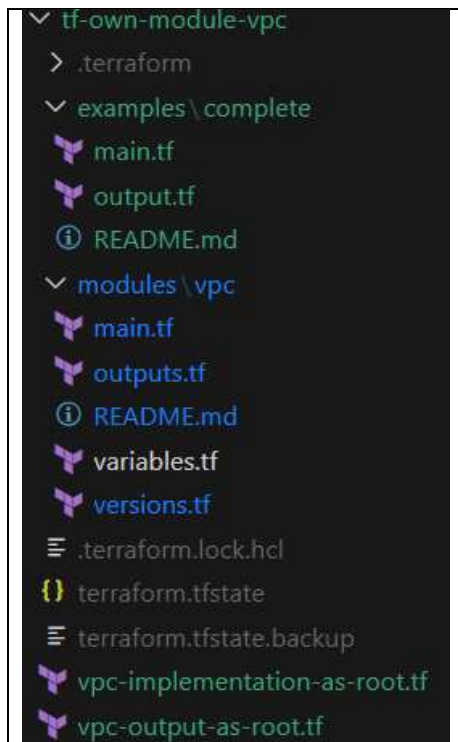
Add the necessary LICENSE file, which you can generate while creating the repository on GitHub.

Now, create a new folder named examples inside the tf-own-module-vpc directory. Within the examples folder, create another subfolder named complete. Inside the complete folder, replicate all the root-level files, such as README.md, main.tf and output.tf.

Copy the contents of ***tf-own-module-vpc/vpc-implementation-as-root.tf*** to ***tf-own-module-vpc/examples/complete/main.tf***

Copy the contents of ***tf-own-module-vpc/vpc-output-as-root.tf*** to ***tf-own-module-vpc/examples/complete/outputs.tf***

Now this is the final folder structure:

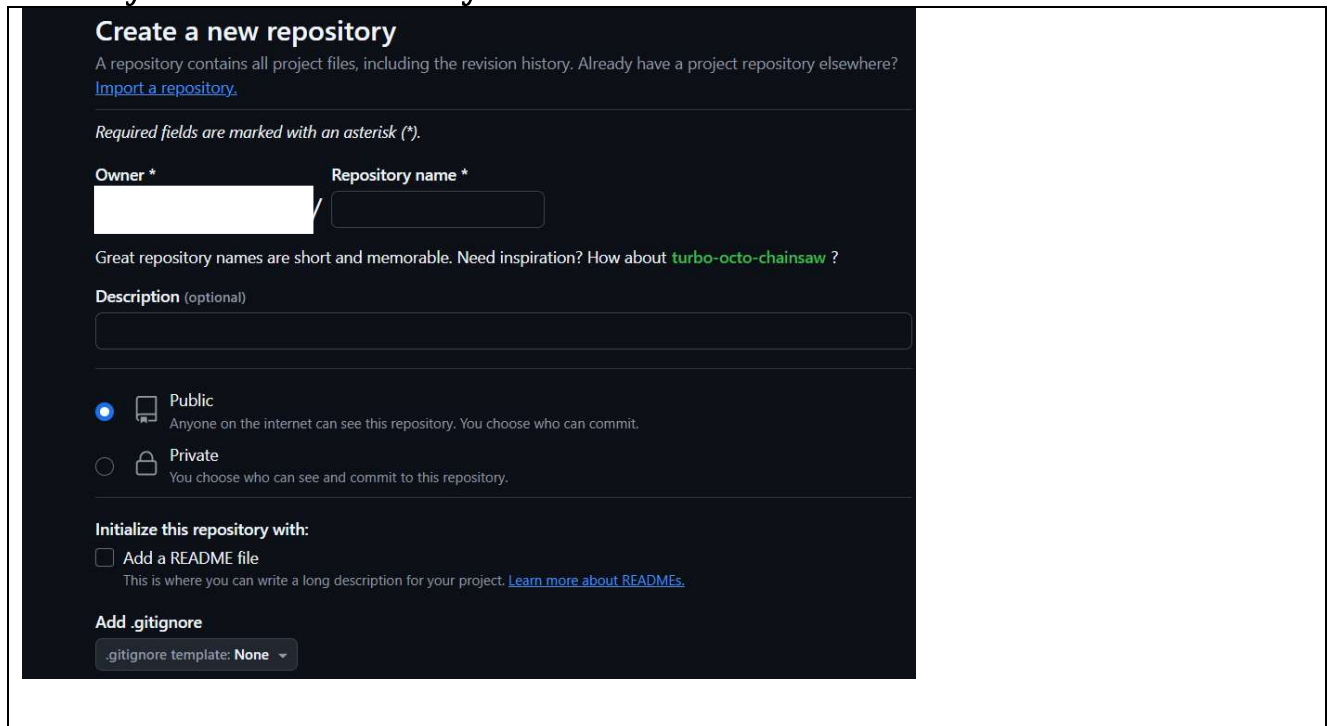


Create a new folder named GitHub-Code-Publisher and copy only the modules and examples directories from tf-own-module-vpc into it.

Log in to your GitHub account, click on the **New** button, and create a new repository.

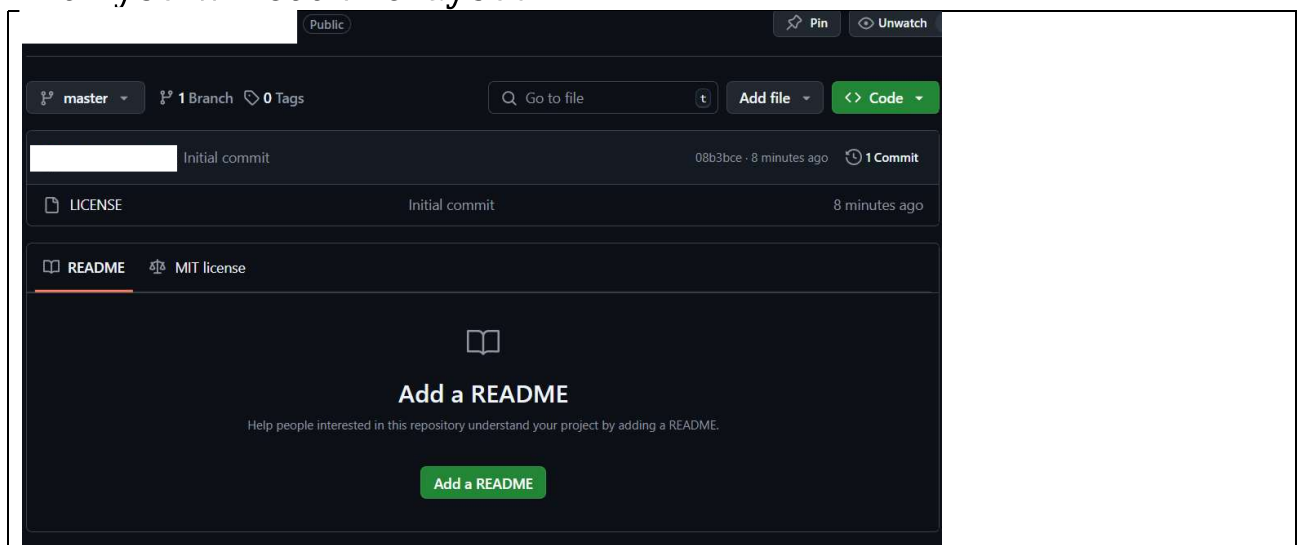


Then you will this this layout



Give the repo name in proper way something like “vpc” and give the MIT license then click on “create repository”

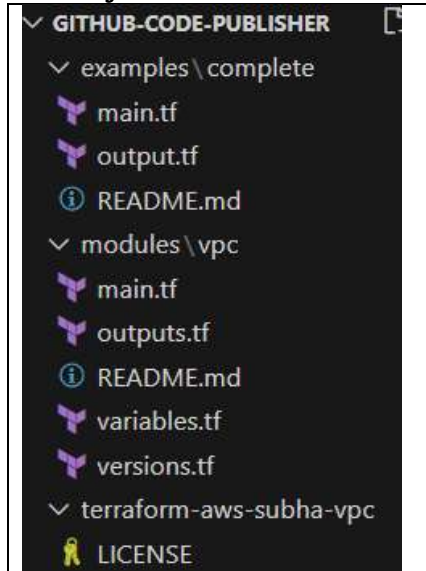
Then you will see this layout



Then click on code and copy the like on the vs code terminal by adding git clone before it

```
PS D:\Downloads\GitHub-Code-Publisher> git clone https://github.com/[redacted]
Cloning into [redacted]
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
PS D:\Downloads\GitHub-Code-Publisher>
```

Now you will see this file structure

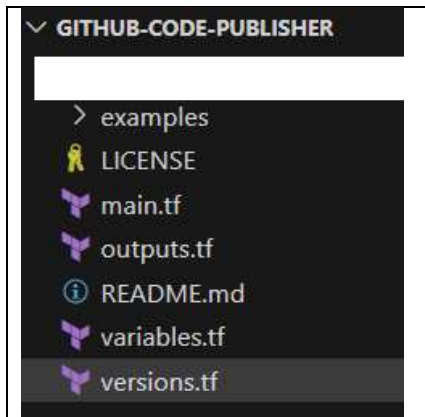


Move all files from the modules/vpc folder (e.g., main.tf, outputs.tf, variables.tf, versions.tf) to the root level of the repository, just outside the modules/vpc directory.

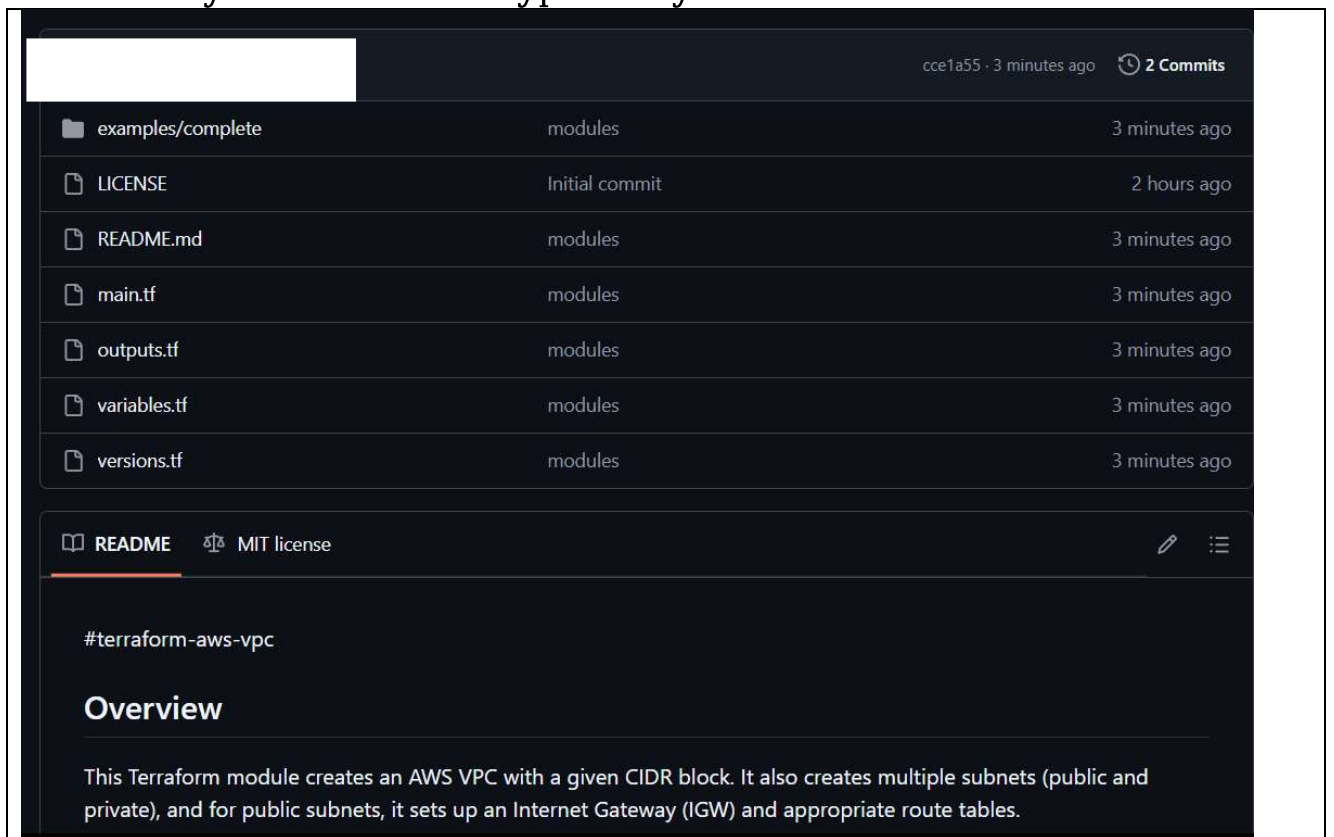
Delete the now-empty modules/vpc folder.

Copy both the files and the examples folder into the terraform-aws-subha-vpc directory.

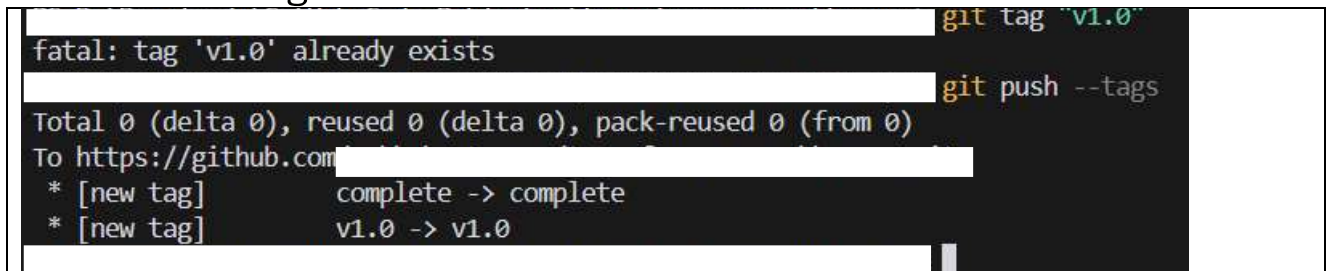
This is the new file structure after editing



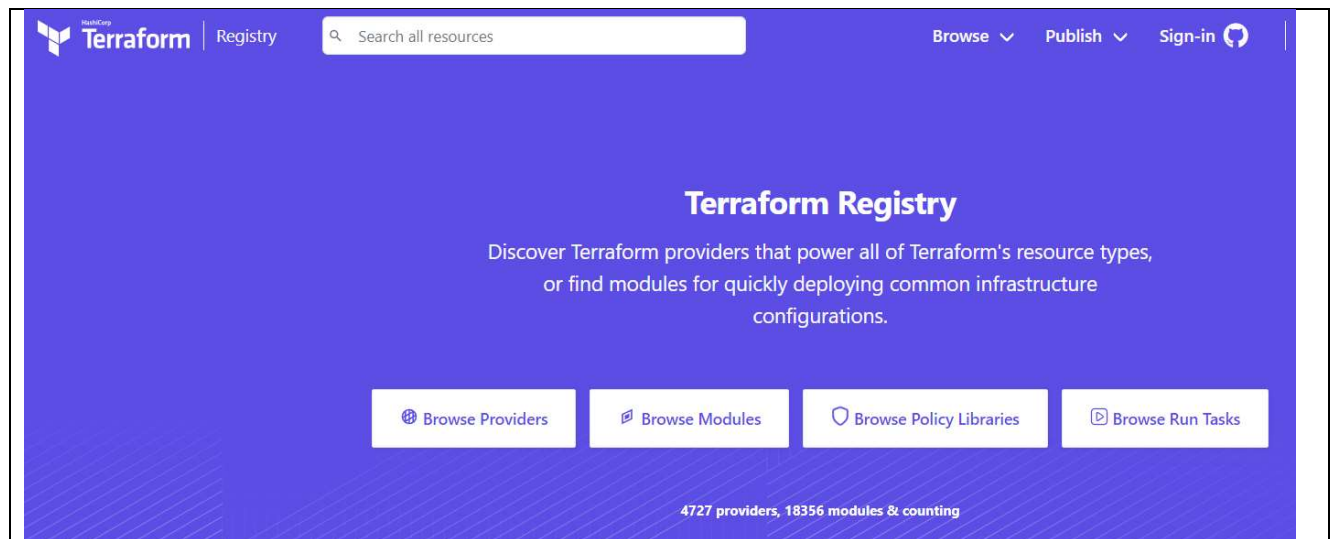
Now add the files and folder to GitHub. After you have added the files and folder you will see this type of layout in GitHub.



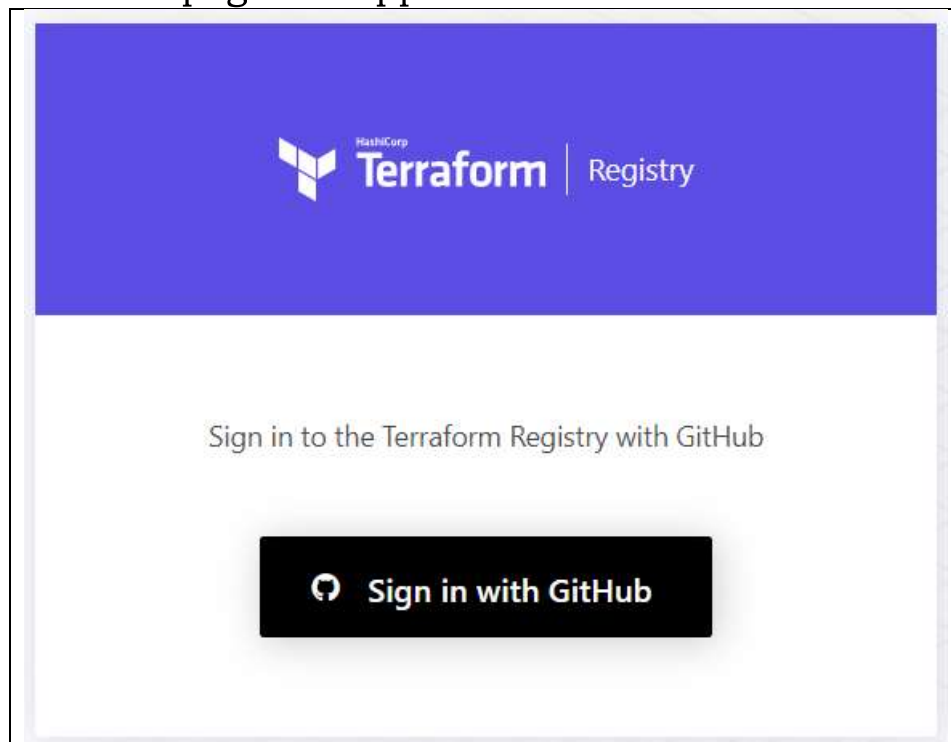
Now add the tag



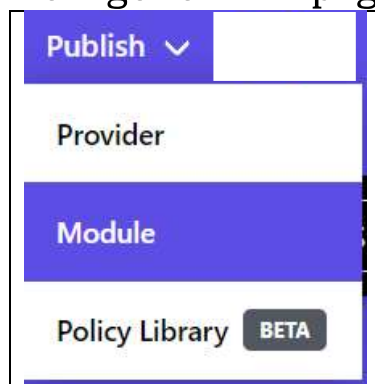
Now go to terraform registry click on sign in option.



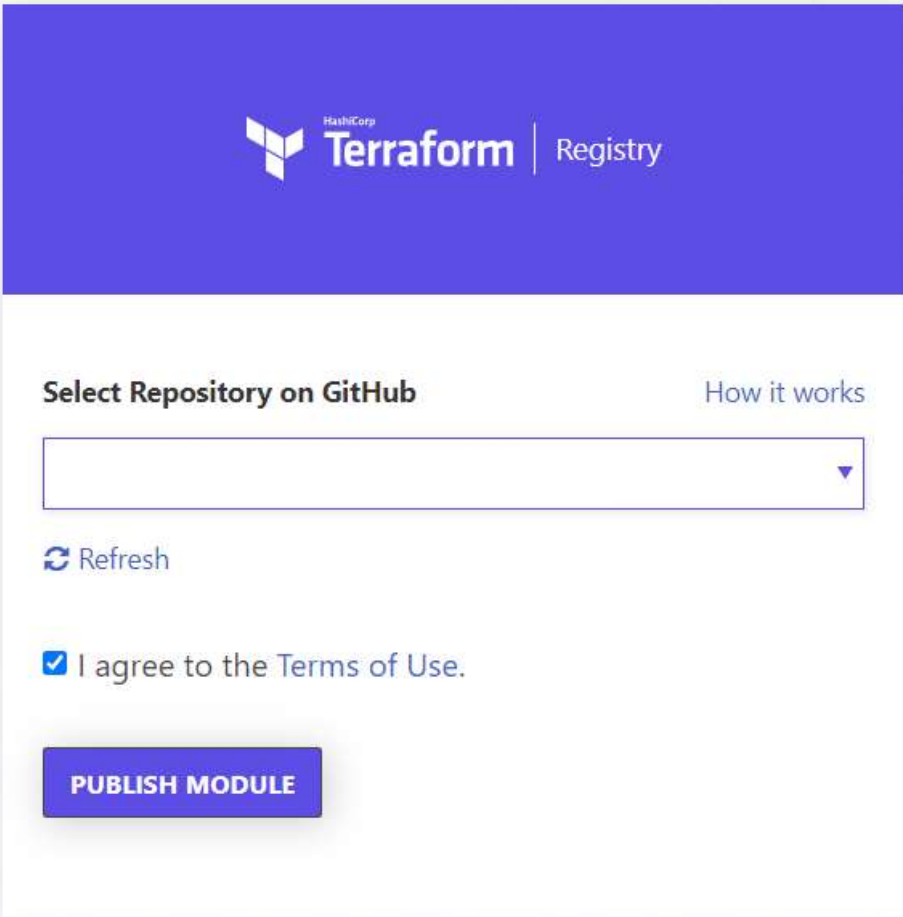
Now this page will appear



Now go to main page, select “publish” and click on module



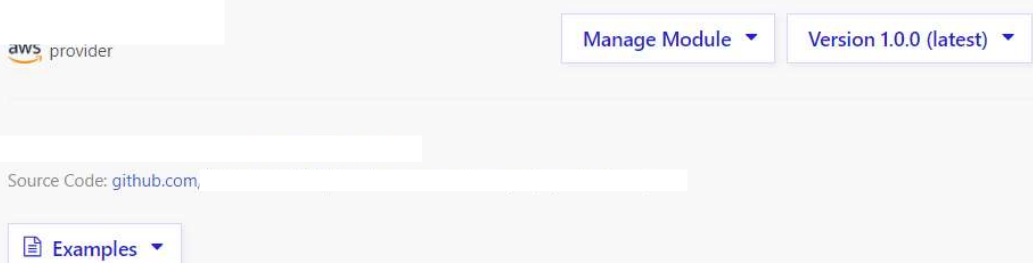
This page will appear



The screenshot shows the Terraform Registry interface for publishing a module. At the top is a blue header with the HashiCorp Terraform logo and the word 'Registry'. Below the header, the main content area has a white background. It starts with the text 'Select Repository on GitHub' and a link 'How it works'. There is a text input field for the repository name, a 'Refresh' button with a circular arrow icon, and a checked checkbox labeled 'I agree to the Terms of Use.'. At the bottom is a large blue button labeled 'PUBLISH MODULE'.

Agree to terms and conditions, then “PUBLISH MODULE”

If all the above steps are correct, then you will see this output in terraform module



The screenshot shows the Terraform module page for the 'aws' provider. The header includes the 'aws provider' label, a 'Manage Module' button with a dropdown arrow, and a 'Version 1.0.0 (latest)' button with a dropdown arrow. Below the header, there is a text input field for the source code, which is currently empty. At the bottom, there is a button labeled 'Examples' with a document icon and a dropdown arrow.

If you want to use this modules as configuration then please follow
“*Implementation of EC2 instance using terraform modules*”